Introduction to Program Analysis

CSE552 Program Analysis — Lecture 1

Jaemin Hong

Instructor Information

- Name: Jaemin Hong
- Email: `jaemin.hong@unist.ac.kr`
- Website: https://hjaem.info/
- Research Areas: Programming Languages, Static Analysis, Program Transformation
- Office: Room 710-1, Building 106
- Office Hours: Wednesday 10:00–11:00 AM (or by appointment)

Course Information

- Lecture slides will be provided
- Textbook: "Static Program Analysis" by Anders Møller and Michael I. Schwartzbach
  - https://cs.au.dk/~amoeller/spa/
- Reference: "Introduction to Static Analysis: An Abstract Interpretation Perspective" by Xavier Rival and Kwangkeun Yi

Grading

- Midterm Exam: 25%
- Final Exam: 25%
- Assignments: 20%
  - 4 programming assignments (5% each)
- Presentations: 30%
  - Term project and end-of-term presentation
  - Define your own software problem and solve it by designing and implementing a static analysis
  - Details will be provided later

Motivation 1: Program Transformation

- Compiler optimization significantly affects software performance
- GCC -O0 vs -O3: 1.2x–15x speedup[1]
- How can we optimize programs automatically?

[1] Impact of GCC optimization levels in energy consumption during C/C++ program execution (Branco and Henriques, 2015)

Example: Loop-Invariant Code Motion

Original:

```
for (i = 0; i < n; i++) {
  y = x * x;
  print(y);
  *p += 1;
}
```

Optimized:

```
y = x * x;
for (i = 0; i < n; i++) {
  print(y);
  *p += 1;
}
```

Does `x * x` evaluate to the same value in every iteration? If so, we can move it outside the loop

Program Transformation: Key Insight

- Semantics-preserving transformation for optimization or refactoring requires understanding program behavior
- We need to know what the program does
- This is where program analysis comes in

Motivation 2: Bug Finding

Software bugs can lead to:

- Significant financial losses
- Security breaches
- Loss of life

We need automated techniques to find bugs before they cause damage

Historical Bug Example: Ariane 5

- Ariane 5 rocket failure (1996)
- Bug type: Integer overflow
- Impact: Loss of more than $370 million[2]

[2]The Ariane 5 software failure (Dowson, 1997)

Historical Bug Example: Therac-25

- Therac-25 radiation therapy machine (1985–1987)
- Bug type: Data race
- Impact: At least 6 accidents, resulting in death or serious injury[3]

[3] An investigation of the Therac-25 accidents (Leveson and Turner, 1992)

Historical Bug Example: Heartbleed

- Heartbleed bug (2014)
- Bug type: Buffer overflow
- Impact: Estimated cost of $500 million[4]

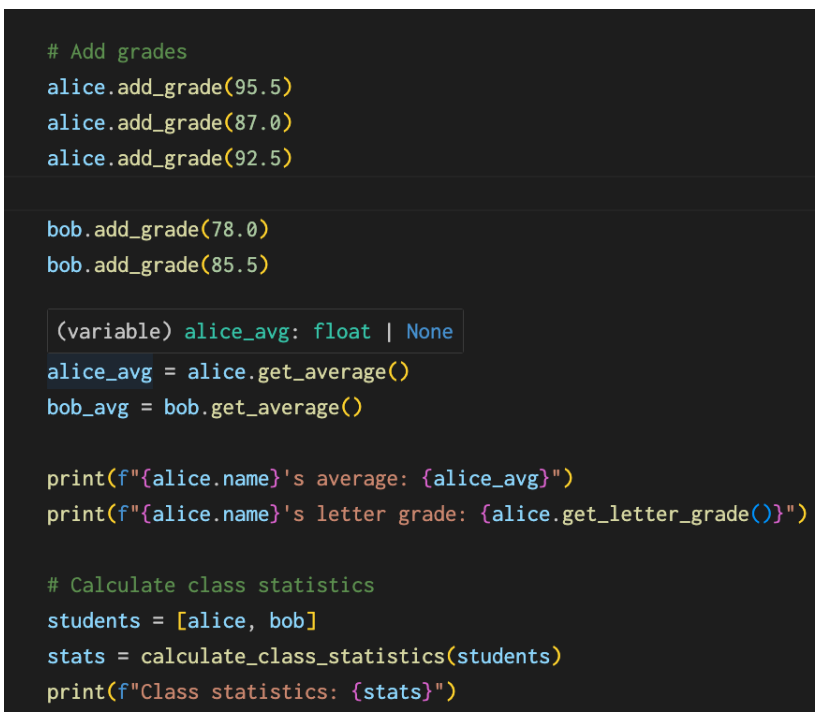
[4] Ten most expensive bugs in history (part 2) (Sixsentix, 2024)

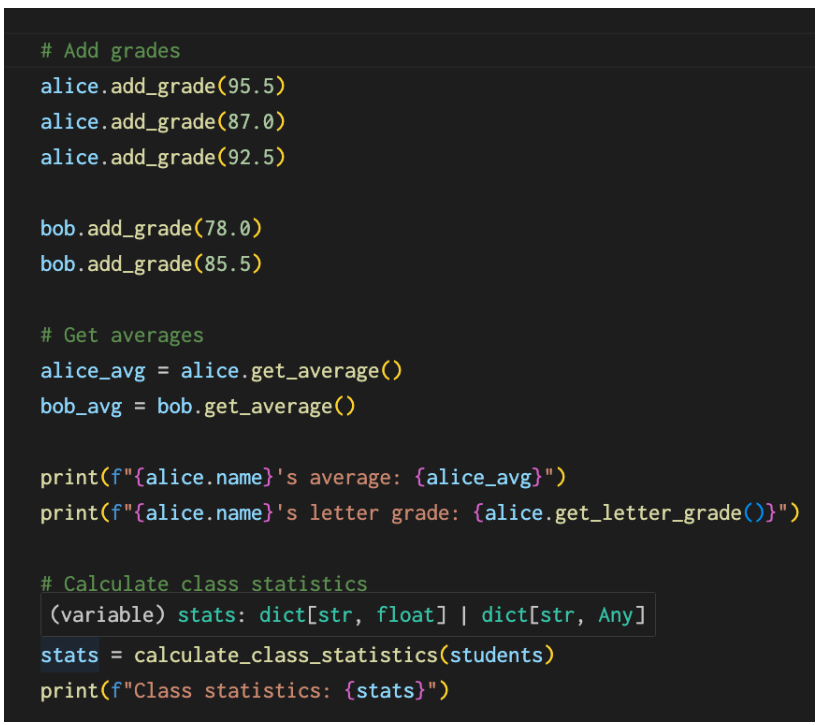
Bug Finding Example

- Can this divisor expression evaluate to zero? If so, we have a division by zero error
- This is a question that program analysis can answer

Motivation 3: Program Comprehension

- Modern development process heavily utilizes IDEs or editor plugins
- Example: "mouse hover" to show the type of an expression





Program Analysis

A collection of techniques to automatically figure out the properties of given programs

Essential for:

- Program transformation
- Bug finding
- Program comprehension

Static vs Dynamic Analysis

- Static: without executing the program
- Dynamic: by executing the program

Focus of This Course: Static Analysis

- We will focus on static analysis techniques
- Why static analysis?

Static Analysis: Advantages

- Ensures termination
  - Execution may not terminate
- Does not require concrete inputs
  - Execution requires concrete inputs
- Can deal with partial programs
  - Execution requires complete programs

Static Analysis: Limitations

- Cannot be both sound and complete for non-trivial properties
- We must choose: soundness or completeness?

Soundness and Completeness

Soundness: If the analysis says a certain behavior is impossible, then it is indeed impossible

- Overapproximates possible behaviors
- No false negatives

Completeness: If the analysis says a certain behavior is possible, then it is indeed possible

- Underapproximates possible behaviors
- No false positives

Example: Soundness vs Completeness

```
if (...) {
  x = 1;
} else {
  x = 2;
}
return x;
```

Sound analysis may say:

- "x is 1 or 2" ✓
- "x is a positive integer" ✓
- "x is any integer" ✓
- but NOT "x is 1" ×

Complete analysis may say:

- "x is 1" ✓
- "x is 2" ✓
- "x is 1 or 2" ✓
- but NOT "x is a positive integer" ×

The Ideal: Sound and Complete

Ideally, we want a sound AND complete analysis:

- If it says a behavior is impossible $\Rightarrow$ it is indeed impossible
- If it says a behavior is possible $\Rightarrow$ it is indeed possible
- No false negatives AND no false positives

But is this possible?

Rice's Theorem

Rice's Theorem: Any non-trivial property of the behavior of programs in a Turing-complete language is undecidable[5]

[5] Classes of recursively enumerable sets and their decision problems (Rice, 1953)

Rice's Theorem: Example

- Solving an analysis problem is at least as hard as solving the halting problem, which is undecidable

```
if (...) {
  f();
  x = 1;
} else {
  x = 2;
}
return x;
```

- x can be 1 only when `f()` terminates
- Determining this requires solving the halting problem
- We must choose either soundness or completeness

Soundness vs Completeness: Which to Choose?

The choice depends on the application

Let's consider two applications:

1. Semantics-preserving transformation
2. Bug finding

For Semantics-Preserving Transformation

```
if (x == 1) { ... } else { ... }
```

- If a complete analysis says "x is 1":
  - Only with this information, we CANNOT remove the `else` branch
  - It might miss the case where x is not 1
- If a sound analysis says "x is 1":
  - x can have no other value
  - We CAN safely remove the `else` branch

Conclusion: We need sound analysis

For Bug Finding

- A sound analysis:
  - Allows proving the absence of bugs
  - But often suffers from false alarms
- A complete analysis:
  - Never produces false alarms
  - But may miss some bugs
- Dynamic analyses are complete:
  - "Program testing can be used to show the presence of bugs, but never to show their absence."[6]
- To complement dynamic analyses, people often design sound static analyses

[6] The humble programmer (Dijkstra, 1972)

Designing Sound Analyses: Challenge

- A sound analysis is easy to implement if soundness is the only goal
  - An analysis that says "every behavior is possible" is trivially sound
  - But it is useless
- To be useful, a sound analysis should be precise as well

Precision

- An analysis is more precise if the set of computed possible behaviors is smaller
- Example (from more to less precise):
  - "x is 1 or 2" (most precise)
  - "x is a positive integer"
  - "x is any integer" (least precise)
- This coincides with the usual use of the term "precision":
  - precision = true positives / all positives

Goals in Static Analysis Design

Minimal goals:

- Termination
- Soundness

Additional goals (to make it useful):

- Efficiency
- Precision

Trade-off: Often exists between efficiency and precision

Course Objectives

- Understand existing static analysis techniques
  - Type analysis, interval analysis, Andersen-style pointer analysis, etc.
- Understand frameworks for designing new static analyses
  - Unification, monotone framework, cubic solver
- Reason about the soundness of static analyses
  - Abstract interpretation

Learning in the LLM Era

Quote from Chris Lattner (creator of LLVM, Clang, Swift, MLIR):[7]

> *"The scarce skills become choosing the right abstractions, defining meaningful problems, and designing systems that humans and AI can evolve together."*
>
> *"Engineers should clarify intent with rigor, validate outcomes with tests, and improve their design."*

[7] https://www.modular.com/blog/the-claude-c-compiler-what-it-reveals-about-the-future-of-software

LLMs and This Course

- LLMs are already good at implementing PL techniques, including static analysis
- **You are allowed and encouraged to use LLMs** for assignments and project
- Focus on:
  - Understanding important ideas in static analysis
  - Designing high-level structures
  - Finding effective ways to manage LLMs
  - Validating the results of LLMs

Summary

- Program analysis automatically figures out program properties
- Static analysis (without execution) vs Dynamic analysis (with execution)
- Static analysis advantages: termination, no inputs needed, partial programs
- Static analysis limitations: cannot be both sound and complete (Rice's theorem)
- We focus on sound analyses with high precision